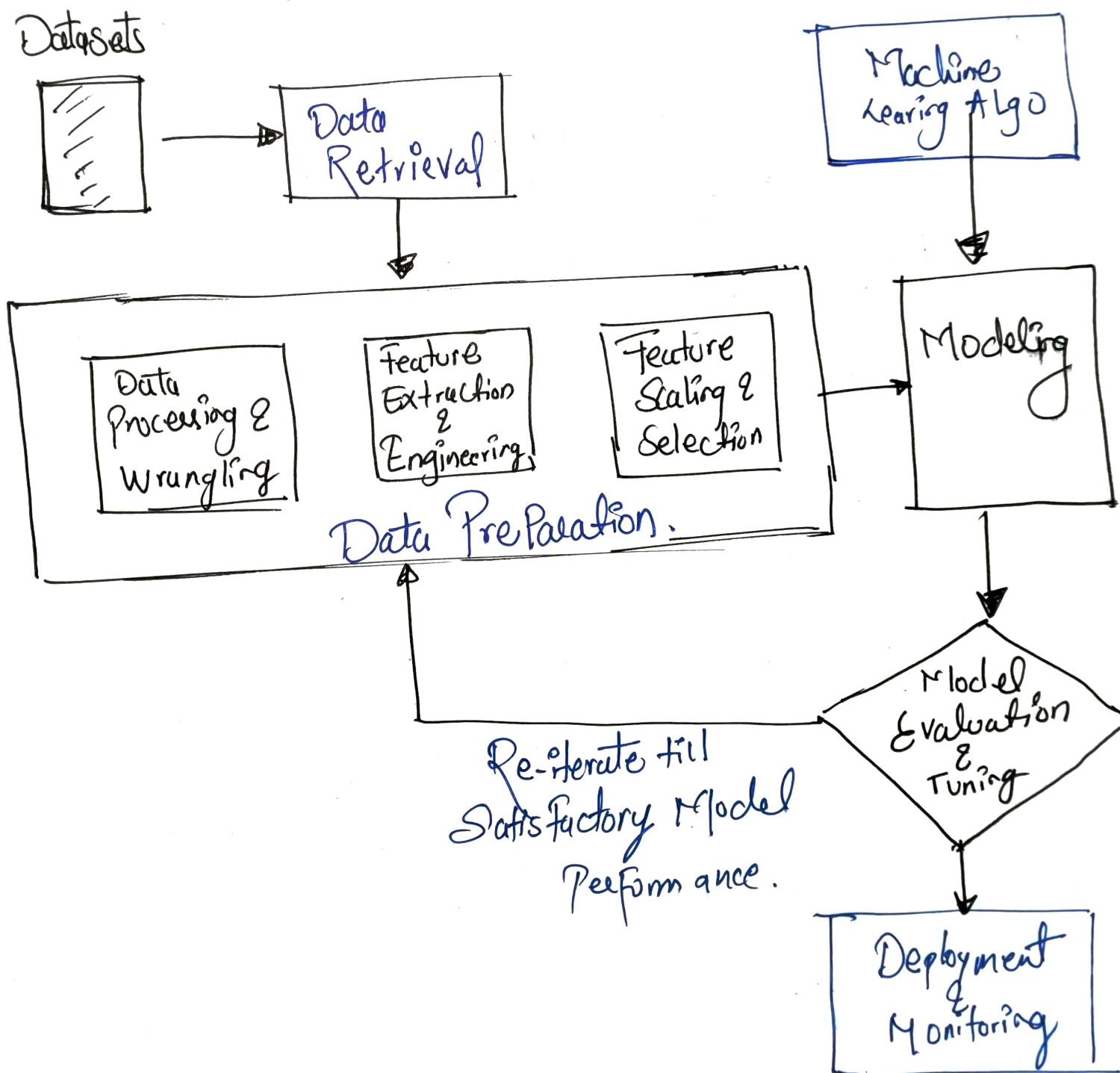


29-July

Feature Engineering :-

Feature engineering is the process of using domain knowledge to extract features from raw data. These features can be used to improve the performance of ML algo.



Feature Engineering

using PCA

Feature Extraction

Feature Transformation

Feature Construction

Feature Selection

Missing Value Imputation

Feature Scaling

Outlier Detection

Handling Categorical Features

Feature Scaling → Data Preprocessing technique

→ used to transform numerical variables into a Common, Comparable Scale without changing the underlying relationships in the data

In raw data features

vary differently e.g. age range 0 - 100

Annual Salary → \$10,000 - \$500,000

→ Feature Scaling prevents variable with larger mathematical values from naturally dominating the model.

Why Important?

→ Prevents Feature bias: Ensures all numerical variables contribute equally to final prediction.

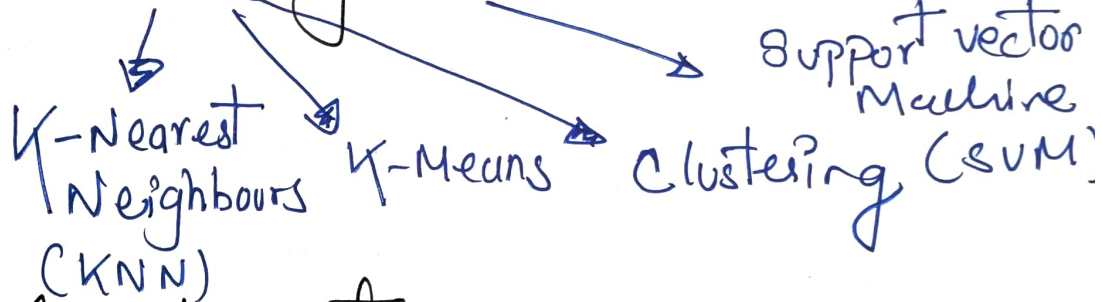
→ Speeds up Optimization: Helps gradient algorithms converge much faster → Reducing training time

→ Balances Distance Calculations: Essential for Algorithms relying on data point distances, preventing skewed spatial calculations.

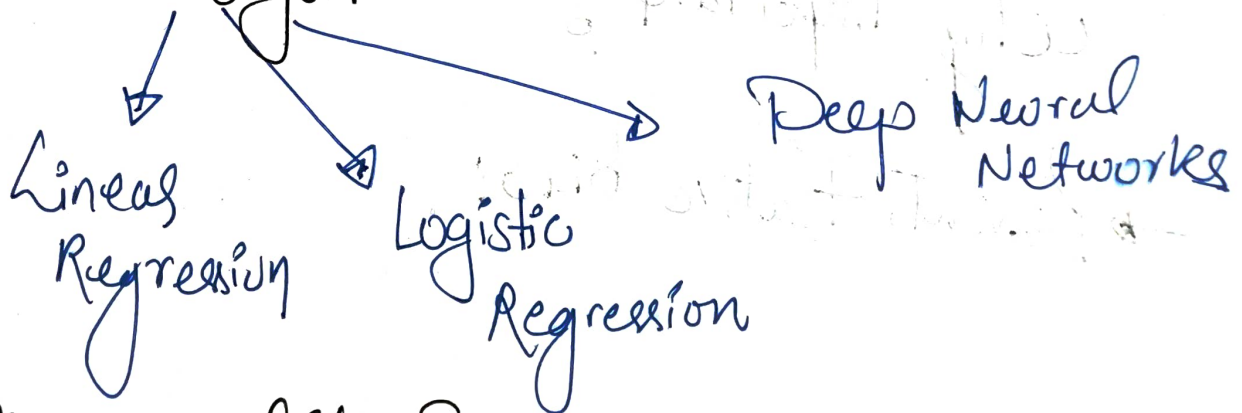
→ Improves Model Accuracy :- Improve accuracy for Scale - Sensitive ML algorithm.

* Apply Scaling For :-

1) Distance-based algorithms:



2) Gradient-descent algorithms:

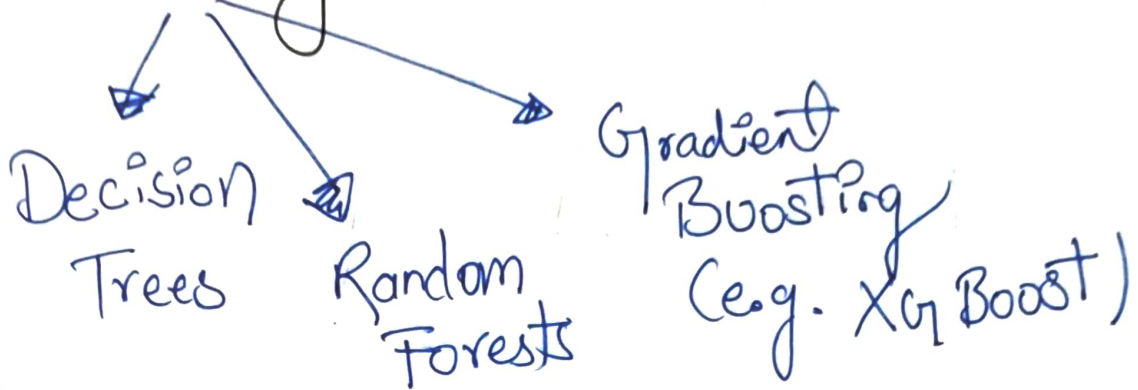


3) Dimensionality Reduction:

e.g. Principal Component Analysis (PCA).

Skip Scaling for & (No Impact by Scaling)

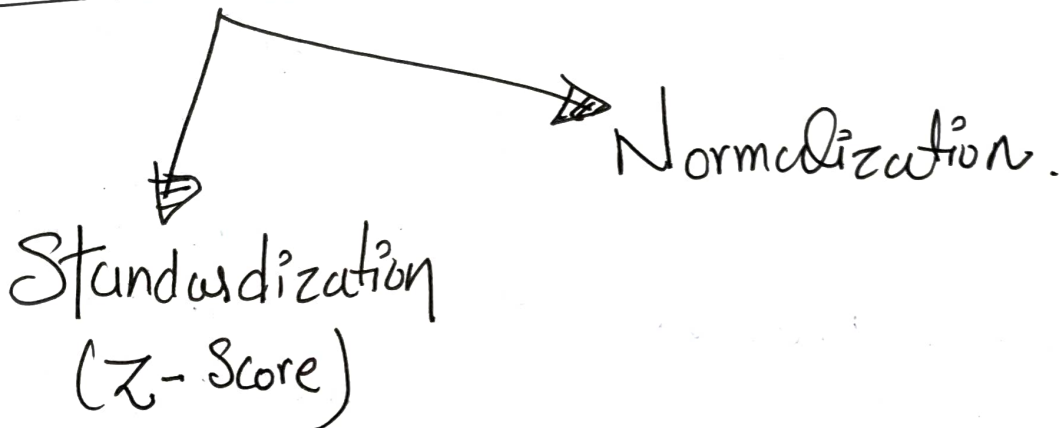
- Tree-Based algorithms:



"Split data based on single-feature thresholds are entirely unaffected by relative feature scale".

Scaling

Technique:



Standardization (also called Z-score normalization)

→ Scaling technique that transform data in a way that it has a $\boxed{\text{Mean}(\mu) = 0}$ and $\boxed{\text{Standard deviation}(\sigma) = 1}$

Goal: we want to transform an existing dataset so that its ^{new} mean is exactly 0 and its new $\sigma = 1$ (variance)

Step 1: Setup a linear Transformation.

↳ basic
Any "shift and scaling" of data can be written as
Linear Equation.

Let X be original variable (with mean μ and std σ)

We want to find a new variable Z using two unknown constants a and b :

$$Z = aX + b \quad \text{--- (1)}$$

we want to find the values of 'a' and 'b' such that

1) The expected value (mean) of Z is 0
i.e. $E(Z) = 0$

2) The variance of Z is 1 i.e.
 $Var(Z) = 1$

Step 2: Solve for a using Variance
apply Variance operator
 $Var(Z) = Var(ax + b)$

$$\therefore Var(ax + b) = a^2 Var(X)$$

$$Var(Z) = a^2 Var(X)$$

$$\therefore Var(X) = \sigma^2$$

$$Var(Z) = a^2 \sigma^2$$

as we want $Var(Z) = 1$ so,

$$1 = a^2 \sigma^2$$

$$\boxed{a = \frac{1}{\sigma}} \quad (ii)$$

Step 3: Solve for b using the mean
apply the expectation (mean) operators
to eqⁿ (i)

$$E(z) = E(ax + b)$$

∴ laws of Expectation: $E[ax + b] = aE(x) + b$

$$E(z) = aE(x) + b$$

∴ by definition $E(x) = \mu$ (Mean)

$$E(z) = a\mu + b$$

also we want Mean to be zero i.e.
 $E(z) = 0$

$$0 = a\mu + b$$

From second Equation

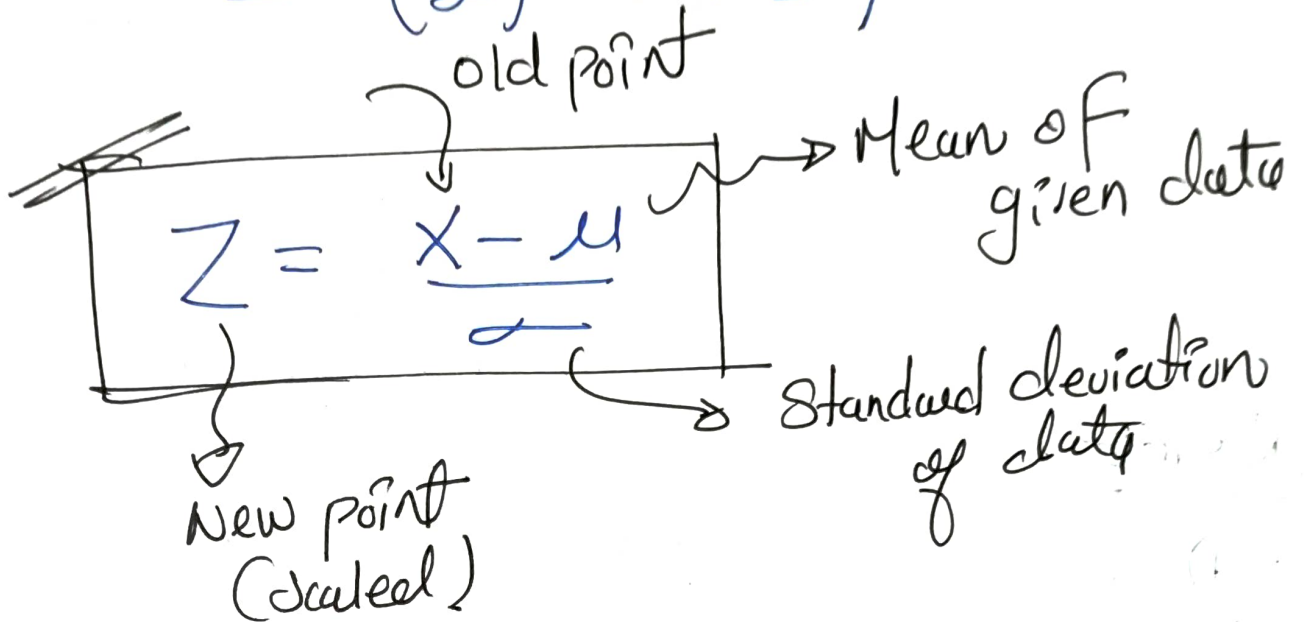
$$0 = a \times \frac{1}{\sigma} + b$$

$$\boxed{b = -\frac{\mu}{\sigma}} \quad \text{--- (iii)}$$

Putting (i) & (ii) in (i)

$$Z = ax + b$$

$$Z = \left(\frac{1}{\sigma}\right)x + \left(-\frac{\mu}{\sigma}\right)$$



How This Transformation
Alters the Shape of
Non-Normal Data?

Misconception = In ML Standardization turns
non-normal (skewed) data
into a perfect, bell-shaped
normal distribution.
— If does not

Standardization is Linear transformation

- It shifts and scale data
- ① but completely Preserve the original
 - ② Shape, relative distances, and
 - ③ Skewness of data

NOTE:-

1) The shape is locked: if data is right skew.

↓ After standardization
right skew

2) The X-axis Change: After scaling axis change

3) Outliers stay Outliers: Extrem values do not get compressed or hidden → They simply get converted into high Z-score.

4) If machine learning algo strictly requires perfectly normal (Gaussian) bell curves, standardization is not enough.

Then Need non-Linear transformations

E.g 1) Log Transformation

2) Box-Cox Transformation.

Why Specifically

Mean = 0 and SD = 1 ?

We force data to have $\mu = 0$ and $\sigma = 1$ because it creates a perfectly balanced "Mathematical Sandbox" for ML Algo.

1) why Mean (μ) = 0 ? (The Anchor of Balance)

a) Symmetric Gradient Step:-

In optimization algo like gradient descent

If all inputs are positive (age, height etc)



Gradients for Model's weights will
Share same sign (either all +ve or -ve)
during an update step.



This forces GD algo to zig-zag
wildly inefficiently to find best
weights.

Centering the
data at '0'



Scaled to
 $\mu=0$

ensures mix of +ve and
negative inputs

→ allowing for smooth,
direct path to
Minimum error



Finds the best
weights
effectively!

b) Neutral Starting
Ground.

In Neural Networks, weights are initialized
randomly around 0.



If input data is also centred at 0

↓
the initial Mathematical Outputs
of the neurons stay small and
Controllable

↓
This Prevents Exploding gradients
at very beginning of training

2) why Standard Deviation $(\sigma) = 1$?

a) Destroys unit Tyranny :-

If Feature A is "Salary" (Spreads across thousands)
and Feature B is "House Size" (Spreads b/w 1 & 5)

↓
An algorithm that calculates Euclidean
distance (KNN or SVM)
will completely ignore house size.

↓
Forcing standard deviation (σ) of both to be
1 means "1-unit change" means
exact same thing for salary & house size

b) Regularization

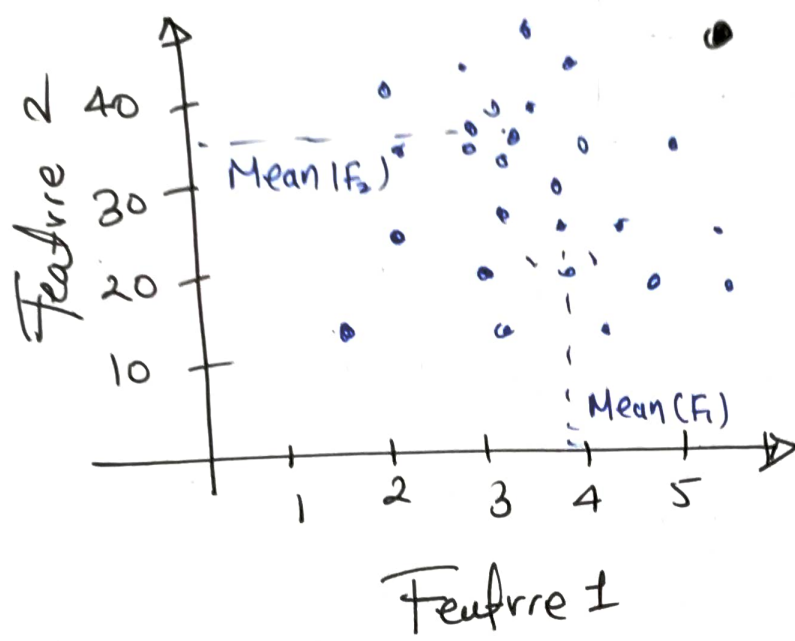
Fairness:

Techniques like Ridge (L_2) or Lasso (L_1) Regression add penalty to model based on the size of weights

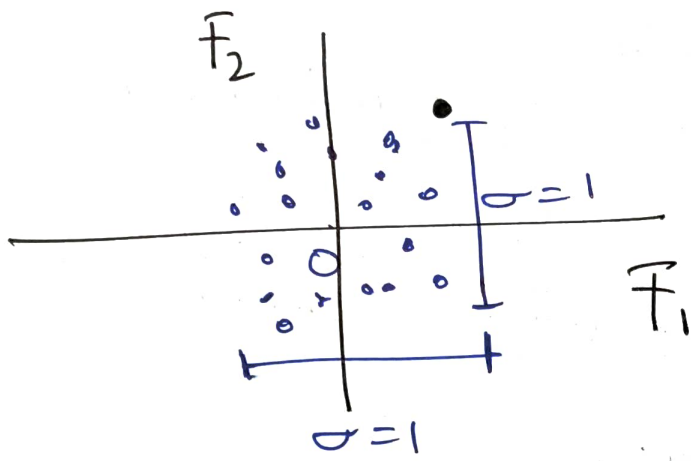
↓
If features aren't on same Variance Scale, Model will unfairly Penalize feature with naturally Smaller raw Values.

↓
Scaling $\sigma = 1$ ensures every feature faces ^{exact} same penalty rules.

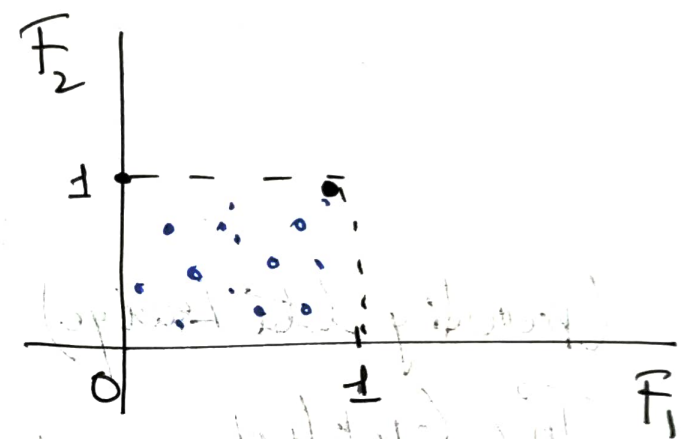
Plots:-



After Standardization



After Normalization



$$Z = \frac{X - \mu}{\sigma}$$

NOTE:-

- 1) If $X > \mu \rightarrow$ data above avg
If $X < \mu \rightarrow$ data below avg $\sigma < 1$
- 2) Dividing (σ) $\rightarrow$ Squashes or Stretches
the data spread so that $\pm$ unit of distance means same thing across all features.
- 3) Preserves Outliers:

Extreme values still exist as high Z-score rather than being compressed into a tiny, unreadable fractional space.

(Preventing data Leakage)

The Golden Rule

Data $\rightarrow$ Train
 $\downarrow$ Test.

1) Fit and Transforms ~~on~~ The Train set:

Calculate μ and σ using Only train data, then Scale it

2) Transform the Test set: Use exact same μ and σ from the train set to Scale test set

"Never recalculate a new mean or standard deviation from test data."

Doing so introduces data leakage. ϵ Feature Inconsistency.
Giving Model unrealistic hints about Feature."

$$\left[\begin{array}{l} \text{train } X - \bar{X} \\ \text{train } X - \bar{X} \end{array} \right] = \left[\text{train } X \right]$$